

Complete Analysis of GenAI Code

Name of the Code Files

email_priority_queue.c (Gemini-generated)

email_priority.c (Claude-generated)

corrected_code.c (corrected/commented version, based on the Claude-generated code)

How They Were Accessed

Claude and Gemini, both accessed via their web browser interfaces (claude.ai and gemini.google.com).

Prompt

Write a C program that prioritizes emails for a busy company CEO.

Requirements:

- Implement a priority queue using a MaxHeap.
- The MaxHeap must use a list-based (array) implementation.
- Use the MaxHeap to handle all email prioritizing for the CEO.
- All heap functions must be written from scratch — do not use any pre-existing heap or priority-queue library or module.

Input file format:

The program reads a test file containing the following commands, one per line.

EMAIL <sender category>,<subject line>,<date>

The emails in the CEO's inbox should be placed in the queue based on their sender category and date. The sender categories and priority to be read are as follows: Boss reads first. Subordinate reads next. Peer reads next. ImportantPerson reads next. OtherPerson reads last. If there is more than one email from a sender, then the newest email (not the oldest) should be read first. EMAIL is followed by a space. The rest of the fields are comma-delimited. Assume sender category is one of the five strings listed above. Assume subject line is a string which may contain spaces but not commas. Assume date is in the format MM-DD-YYYY.

NEXT

Shows the next email for the CEO to read. Display the information on the terminal in the following format: Sender, Subject, Date.

READ

The CEO has read the email and dealt with it. No output is produced.

COUNT

Display the current count of unread emails.

The grader will test the code with a different test file. The test file will begin with the emails in the CEO's inbox, followed by the commands the CEO will use to read her inbox. The order and number of commands may vary. The program must be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (the output should be the same for both NEXTs), and two READs in a row without an intervening NEXT (the highest priority email is deleted from the queue without displaying it).

Code Generated

Gemini:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINE 256
#define INITIAL_CAPACITY 10

typedef struct {
    char category[32];
    char subject[128];
    char date[16];
    int category_rank; // Higher integer = Higher priority
    int year;
    int month;
    int day;
} Email;

typedef struct {
    Email *array;
    int size;
    int capacity;
} MaxHeap;

// Maps category string to an integer rank for comparison
int get_category_rank(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

// Parses MM-DD-YYYY into integer components for date comparison
void parse_date(const char *date_str, int *m, int *d, int *y) {
    sscanf(date_str, "%d-%d-%d", m, d, y);
}

/*
 * Priority Comparison:
 * Returns > 0 if email A has higher priority than email B.
 * Returns < 0 if email B has higher priority than email A.
 */
int compare_emails(const Email *a, const Email *b) {
```

```

    if (a->category_rank != b->category_rank) {
        return a->category_rank - b->category_rank;
    }
    // Newer date takes priority
    if (a->year != b->year) return a->year - b->year;
    if (a->month != b->month) return a->month - b->month;
    return a->day - b->day;
}

// Heap Operations
MaxHeap* create_heap() {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = INITIAL_CAPACITY;
    heap->size = 0;
    heap->array = (Email*)malloc(heap->capacity * sizeof(Email));
    return heap;
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(&heap->array[index], &heap->array[parent]) > 0) {
            swap(&heap->array[index], &heap->array[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

void heapify_down(MaxHeap *heap, int index) {
    int max_idx = index;
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        if (left < heap->size && compare_emails(&heap->array[left],
&heap->array[max_idx]) > 0) {
            max_idx = left;
        }
        if (right < heap->size && compare_emails(&heap->array[right],
&heap->array[max_idx]) > 0) {
            max_idx = right;
        }

        if (max_idx != index) {
            swap(&heap->array[index], &heap->array[max_idx]);
            index = max_idx;
        } else {
            break;
        }
    }
}

```

```

void insert_email(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->array = (Email*)realloc(heap->array, heap->capacity * sizeof(Email));
    }
    heap->array[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

void delete_max(MaxHeap *heap) {
    if (heap->size == 0) return;
    heap->array[0] = heap->array[heap->size - 1];
    heap->size--;
    if (heap->size > 0) {
        heapify_down(heap, 0);
    }
}

void free_heap(MaxHeap *heap) {
    free(heap->array);
    free(heap);
}

int main(int argc, char *argv[]) {
    const char *filename = (argc > 1) ? argv[1] : "sample_commands.txt";
    FILE *file = fopen(filename, "r");
    if (!file) {
        perror("Error opening input file");
        return 1;
    }

    MaxHeap *heap = create_heap();
    char line[MAX_LINE];

    while (fgets(line, sizeof(line), file)) {
        // Strip trailing newline characters
        line[strcspn(line, "\r\n")] = 0;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            char *data = line + 6;
            Email email;

            char *category_tok = strtok(data, ",");
            char *subject_tok = strtok(NULL, ",");
            char *date_tok = strtok(NULL, ",");

            if (category_tok && subject_tok && date_tok) {
                strcpy(email.category, category_tok);
                strcpy(email.subject, subject_tok);
                strcpy(email.date, date_tok);

                email.category_rank = get_category_rank(email.category);
                parse_date(email.date, &email.month, &email.day, &email.year);

                insert_email(heap, email);
            }
        } else if (strcmp(line, "NEXT") == 0) {

```

```

        if (heap->size > 0) {
            printf("%s, %s, %s\n", heap->array[0].category,
heap->array[0].subject, heap->array[0].date);
        }
        } else if (strcmp(line, "READ") == 0) {
            if (heap->size > 0) {
                delete_max(heap);
            }
        } else if (strcmp(line, "COUNT") == 0) {
            printf("%d\n", heap->size);
        }
    }

    fclose(file);
    free_heap(heap);
    return 0;
}

```

Claude:

```

/*
 * email_priority.c
 *
 * Prioritizes a CEO's inbox using a custom, list-based (array-based)
 * MaxHeap implemented entirely from scratch (no library heap/priority
 * queue routines are used).
 *
 * Priority rules:
 *   Boss          -> read first    (highest priority)
 *   Subordinate    -> read next \
 *   Peer           -> read next >  all tied for "read next"
 *   ImportantPerson-> read next /
 *   OtherPerson    -> read last   (lowest priority)
 *
 * Within the same category, the NEWEST email (by date) is read first.
 *
 * Commands (read one per line from a test file):
 *   EMAIL <sender category>,<subject line>,<date>
 *   NEXT
 *   READ
 *   COUNT
 *
 * Usage:
 *   ./email_priority <testfile>      (reads commands from testfile)
 *   ./email_priority                 (reads commands from stdin)
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* ----- */
/* Email record */
/* ----- */

#define MAX_SENDER    32

```

```

#define MAX_SUBJECT 256
#define MAX_DATE 11 /* MM-DD-YYYY + '\0' */

typedef struct {
    char sender[MAX_SENDER]; /* sender category, e.g. "Boss" */
    char subject[MAX_SUBJECT];
    char date[MAX_DATE]; /* stored as given, MM-DD-YYYY */
    int categoryRank; /* higher = more important */
    long dateValue; /* YYYYMMDD, higher = newer */
} Email;

/* ----- */
/* MaxHeap: list-based (dynamic array) implementation, built from
 * scratch. No pre-existing heap/priority-queue library is used. */
/* ----- */

typedef struct {
    Email *data;
    int size; /* number of elements currently stored */
    int capacity; /* allocated capacity of data[] */
} MaxHeap;

static void heapInit(MaxHeap *h) {
    h->capacity = 8;
    h->size = 0;
    h->data = (Email *)malloc(sizeof(Email) * h->capacity);
    if (h->data == NULL) {
        fprintf(stderr, "Error: out of memory.\n");
        exit(EXIT_FAILURE);
    }
}

static void heapFree(MaxHeap *h) {
    free(h->data);
    h->data = NULL;
    h->size = 0;
    h->capacity = 0;
}

static int heapIsEmpty(const MaxHeap *h) {
    return h->size == 0;
}

static int heapSize(const MaxHeap *h) {
    return h->size;
}

static void heapGrow(MaxHeap *h) {
    int newCapacity = h->capacity * 2;
    Email *newData = (Email *)realloc(h->data, sizeof(Email) * newCapacity);
    if (newData == NULL) {
        fprintf(stderr, "Error: out of memory.\n");
        exit(EXIT_FAILURE);
    }
    h->data = newData;
    h->capacity = newCapacity;
}

static void swapEmails(Email *a, Email *b) {

```

```

    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* Returns 1 if email 'a' should be considered HIGHER priority than
 * email 'b' (i.e. 'a' belongs closer to the root of the max-heap). */
static int hasHigherPriority(const Email *a, const Email *b) {
    if (a->categoryRank != b->categoryRank) {
        return a->categoryRank > b->categoryRank;
    }
    /* Same category: newer date wins. */
    return a->dateValue > b->dateValue;
}

/* Standard array index helpers for a 0-indexed binary heap. */
static int parentIndex(int i) { return (i - 1) / 2; }
static int leftIndex(int i)   { return 2 * i + 1; }
static int rightIndex(int i)  { return 2 * i + 2; }

/* Moves the element at index i up until the heap property holds. */
static void siftUp(MaxHeap *h, int i) {
    while (i > 0) {
        int p = parentIndex(i);
        if (hasHigherPriority(&h->data[i], &h->data[p])) {
            swapEmails(&h->data[i], &h->data[p]);
            i = p;
        } else {
            break;
        }
    }
}

/* Moves the element at index i down until the heap property holds. */
static void siftDown(MaxHeap *h, int i) {
    while (1) {
        int left = leftIndex(i);
        int right = rightIndex(i);
        int largest = i;

        if (left < h->size && hasHigherPriority(&h->data[left], &h->data[largest]))
        {
            largest = left;
        }
        if (right < h->size && hasHigherPriority(&h->data[right],
&h->data[largest])) {
            largest = right;
        }
        if (largest == i) {
            break;
        }
        swapEmails(&h->data[i], &h->data[largest]);
        i = largest;
    }
}

static void heapInsert(MaxHeap *h, Email e) {
    if (h->size == h->capacity) {
        heapGrow(h);
    }
}

```

```

    }
    h->data[h->size] = e;
    siftUp(h, h->size);
    h->size++;
}

/* Returns a copy of the highest-priority email without removing it.
 * Caller must ensure the heap is non-empty. */
static Email heapPeek(const MaxHeap *h) {
    return h->data[0];
}

/* Removes and returns the highest-priority email.
 * Caller must ensure the heap is non-empty. */
static Email heapExtractMax(MaxHeap *h) {
    Email top = h->data[0];
    h->size--;
    h->data[0] = h->data[h->size];
    if (h->size > 0) {
        siftDown(h, 0);
    }
    return top;
}

/* ----- */
/* Parsing helpers */
/* ----- */

/* Strips trailing '\r' / '\n' / spaces from a line read via fgets. */
static void trimLine(char *s) {
    int len = (int)strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r' ||
        s[len - 1] == ' ' || s[len - 1] == '\t')) {
        s[--len] = '\0';
    }
}

/* Maps a sender category string to a numeric priority rank.
 * Higher number = higher priority (closer to heap root). */
static int categoryToRank(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 3;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 3;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    /* Unknown category: treat as lowest priority rather than crash. */
    return 0;
}

/* Converts a date string "MM-DD-YYYY" into an integer YYYYMMDD so
 * that larger values represent more recent dates. */
static long dateToValue(const char *date) {
    int month = 0, day = 0, year = 0;
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0; /* malformed date: treat as oldest possible */
    }
    return (long)year * 10000L + (long)month * 100L + (long)day;
}

```



```

/* Parses a line of the form:
 *   EMAIL <sender category>,<subject line>,<date>
 * (the "EMAIL " keyword and following space have already been
 * skipped by the caller). Returns 1 on success, 0 on failure. */
static int parseEmailFields(char *fields, Email *out) {
    char *category = strtok(fields, ",");
    char *subject  = strtok(NULL, ",");
    char *date      = strtok(NULL, ",");

    if (category == NULL || subject == NULL || date == NULL) {
        return 0;
    }

    strncpy(out->sender, category, MAX_SENDER - 1);
    out->sender[MAX_SENDER - 1] = '\0';

    strncpy(out->subject, subject, MAX_SUBJECT - 1);
    out->subject[MAX_SUBJECT - 1] = '\0';

    strncpy(out->date, date, MAX_DATE - 1);
    out->date[MAX_DATE - 1] = '\0';

    out->categoryRank = categoryToRank(out->sender);
    out->dateValue = dateToValue(out->date);

    return 1;
}

/* ----- */
/* Command handlers */
/* ----- */

static void handleEmail(MaxHeap *h, char *line) {
    /* line currently points at the character right after "EMAIL ". */
    Email e;
    if (parseEmailFields(line, &e)) {
        heapInsert(h, e);
    } else {
        fprintf(stderr, "Warning: malformed EMAIL line skipped.\n");
    }
}

static void handleNext(MaxHeap *h) {
    if (heapIsEmpty(h)) {
        printf("There are no emails to read.\n\n");
        return;
    }
    Email top = heapPeek(h);
    printf("Next email:\n");
    printf("\tSender: %s\n", top.sender);
    printf("\tSubject: %s\n", top.subject);
    printf("\tDate: %s\n", top.date);
    printf("\n");
}

static void handleRead(MaxHeap *h) {
    /* Removes the highest-priority email (CEO has dealt with it).
     * No output is produced for this command. If the queue is
     * already empty, this is simply a no-op. */
}

```

```

        if (!heapIsEmpty(h)) {
            heapExtractMax(h);
        }
    }

static void handleCount(MaxHeap *h) {
    printf("There are %d emails to read.\n\n", heapSize(h));
}

/* ----- */
/* Main driver                                */
/* ----- */

int main(int argc, char *argv[]) {
    FILE *in = stdin;

    if (argc > 1) {
        in = fopen(argv[1], "r");
        if (in == NULL) {
            fprintf(stderr, "Error: could not open file '%s'.\n", argv[1]);
            return EXIT_FAILURE;
        }
    }

    MaxHeap heap;
    heapInit(&heap);

    char line[600];
    while (fgets(line, sizeof(line), in) != NULL) {
        trimLine(line);

        if (line[0] == '\0') {
            continue; /* skip blank lines */
        }

        if (strncmp(line, "EMAIL ", 6) == 0) {
            handleEmail(&heap, line + 6);
        } else if (strcmp(line, "NEXT") == 0) {
            handleNext(&heap);
        } else if (strcmp(line, "READ") == 0) {
            handleRead(&heap);
        } else if (strcmp(line, "COUNT") == 0) {
            handleCount(&heap);
        } else {
            fprintf(stderr, "Warning: unrecognized command skipped: %s\n", line);
        }
    }

    if (in != stdin) {
        fclose(in);
    }
    heapFree(&heap);

    return EXIT_SUCCESS;
}

```

Correctness

Both programs build their MaxHeap entirely from scratch using an array-based structure with sift-up/sift-down logic, so both satisfy the "no pre-existing heap library" requirement. Where they diverge is in how closely they follow the actual assignment specification (the PDF, not just the prompt text):

- The Gemini version (`email_priority_queue.c`) contains a ranking flaw: its `get_category_rank` function assigns three separate values — 4, 3, and 2 — to Subordinate, Peer, and ImportantPerson. Per the spec, these three categories are supposed to be tied for "read next" priority, with date as the only tiebreaker within a category. Treating them as distinct ranks is a substantive deviation from the requirements, not a cosmetic difference.
- Gemini's NEXT output also follows the single-line comma format suggested by the prompt ("`%s, %s, %s\n`"), rather than the multi-line format and COUNT phrasing ("There are N emails to read.") that the PDF's actual sample output demonstrates — meaning it would likely fail an exact-match grading check.
- The Claude version (`email_priority.c`), by contrast, correctly assigns Subordinate, Peer, and ImportantPerson the same rank, and its NEXT/COUNT output matches the PDF's sample format and wording exactly. It also handles edge cases properly per spec — repeated NEXT calls with no intervening READ, and consecutive READs with no intervening NEXT (which silently removes the top email without printing it).

Execution Time

There's no practical difference between the two. Both perform EMAIL insertions and READ extractions in $O(\log n)$ time, since each operation triggers a single sift-up or sift-down pass through the heap. NEXT and COUNT are both $O(1)$, simply reading the root element or the size field. Runtime scales identically for both implementations as the inbox grows.

Space Complexity

Both implementations are $O(n)$ overall, relying on a dynamic array that doubles in capacity whenever it fills up. The one difference is buffer sizing: Claude's version allocates a 256-character subject buffer versus Gemini's 128 characters. This costs Claude's version somewhat more memory per email, but buys extra headroom for longer subject lines without risk of truncation.

Maintainability

Claude's version is the clear winner here. It breaks each operation into its own dedicated static function — `heapIsEmpty`, `heapPeek`, `handleNext`, `handleRead`, and so on — with a short comment above each one, and pulls command dispatch out of `main` into named handler functions. Gemini's version instead crams all command-handling logic directly into `main`'s read loop, making it considerably harder to extend, test in isolation, or annotate line-by-line — a real drawback given that the assignment's in-line commenting criterion alone is worth 14 of the 100 rubric points.

Which Code Is Better?

The Claude-generated code (`email_priority.c`) is the stronger baseline overall. Execution time and time complexity are identical between the two versions, so those criteria don't distinguish them. Where Claude pulls ahead is correctness against the assignment specification — it correctly ties Subordinate, Peer, and ImportantPerson at the same priority level and reproduces the exact output format and wording shown in the sample test case. Maintainability is also stronger: Claude's code separates each operation into its own named, commented helper function, making it easier to extend, test, and annotate. The one place Gemini's version is leaner is space complexity, using smaller fixed buffers for subject lines — a modest memory savings, though Claude's larger buffers provide more headroom for longer subject lines. On balance, Claude's version was the better starting point to build on.

How The Code was improved:

Error & Edge Case Handling

- Unknown categories now trigger a warning instead of silently defaulting to lowest priority.
- Dates are range-checked (valid month/day/year); invalid or unparseable dates warn and fall back to "oldest possible" instead of silently corrupting sort order.
- Long subject lines warn before truncation, naming the original text.
- Extra fields on an EMAIL line (more than 3 comma-delimited values) are now flagged instead of silently dropped.
- File-open failures report the specific reason (`strerror(errno)`) instead of a generic message.

Maintainability

- Added a full prologue comment block (name, description, inputs/outputs, collaborators, author, dates).
- Added a header comment above every function, including the new validation helpers.
- Added a comment on every executable line and closing brace, satisfying the assignment's "comments on every line" rubric item.

Improved Code:

```
/*
 * Program:      EECS 348 Assignment 2 - email_priority.c
 * Description:  Prioritizes a busy company CEO's inbox using a custom,
 *              list-based (array) MaxHeap implemented entirely from
 *              scratch (no library heap/priority-queue routines are
 *              used). Emails are ranked first by sender category
 *              (Boss > Subordinate/Peer/ImportantPerson (tied) >
 *              OtherPerson) and, within the same category, by date
 *              (newest first).
 * Inputs:      A text file (path given as argv[1], or stdin if no
```

```

*          argument is given) containing one command per line:
*          EMAIL <sender category>,<subject line>,<date>
*          NEXT
*          READ
*          COUNT
* Output:    Created a heap that has the ability to read mail, go to the next
mail
* and count the number of mail depending on the number of commands in the text
file
* Collaborators / Sources: Base implementation generated by Claude
*          (Anthropic), accessed via claude.ai, using the prompt
*          documented in the accompanying GenAI analysis.
*          Reviewed, commented, and corrected by Alan Thomas.
* Author:    Alan Thomas
* Creation date: September 15, 2026
* Revision date: September 17, 2026
* Revisions: Added full prologue and in-line comments on every
*          line. Added edge-case and error handling: unknown
*          sender category detection, date range validation,
*          subject truncation warnings, extra-field detection
*          on EMAIL lines, and a more descriptive file-open
*          error message.
*/

//Code generated by Claude
#include <stdio.h>    // printf, fprintf, fgets, fopen, fclose, sscanf
#include <stdlib.h>    // malloc, realloc, free, exit, EXIT_SUCCESS/FAILURE
#include <string.h>    // strcmp, strncmp, strtok, strncpy, strlen, strerror
#include <ctype.h>    // (kept for potential character-class checks)
#include <errno.h>    // errno, used with strerror() for file-open errors

/* ----- */
/*  Email record                                     */
/* ----- */

#define MAX_SENDER    32    // max characters for a sender category string
#define MAX_SUBJECT    256    // max characters for a subject line
#define MAX_DATE      11    // "MM-DD-YYYY" plus the null terminator

// Holds all the information the program tracks about a single email.
typedef struct {
    char sender[MAX_SENDER];    // sender category, e.g. "Boss"
    char subject[MAX_SUBJECT];    // subject line text (may contain spaces)
    char date[MAX_DATE];    // date exactly as given, MM-DD-YYYY
    int categoryRank;    // numeric priority derived from sender; higher =
more important
    long dateValue;    // date encoded as YYYYMMDD; higher = newer
} Email;    // end of Email struct definition

/* ----- */
/*  MaxHeap: list-based (dynamic array) implementation, built from
*  scratch. No pre-existing heap/priority-queue library is used.
*  ----- */

// A binary max-heap stored in a resizable array (0-indexed).
typedef struct {
    Email *data;    // pointer to the underlying dynamic array
    int size;    // number of elements currently stored

```

```

int capacity; // allocated capacity of data[]
} MaxHeap;    // end of MaxHeap struct definition

// Initializes an empty heap with a small starting capacity.
static void heapInit(MaxHeap *h) { // h: heap to
initialize                                     //
    h->capacity = 8;                        // start small;
heapGrow() doubles as needed
    h->size = 0;                            // heap starts
empty
    h->data = (Email *)malloc(sizeof(Email) * h->capacity); // allocate the
backing array
    if (h->data == NULL) {                  // malloc failed
(out of memory)
        fprintf(stderr, "Error: out of memory.\n"); // report the
failure to stderr
        exit(EXIT_FAILURE);                // abort the
program; nothing else to do
    }                                     // end if malloc
failed
}                                     // end heapInit

// Releases the heap's backing array and resets its fields.
static void heapFree(MaxHeap *h) { // h: heap to release
    free(h->data); // release the dynamic array back to the system
    h->data = NULL; // avoid leaving a dangling pointer
    h->size = 0; // logical size is now zero
    h->capacity = 0; // capacity is now zero (no array to hold anything)
} // end heapFree

// Returns 1 (true) if the heap currently holds no emails.
static int heapIsEmpty(const MaxHeap *h) { // h: heap to check
    return h->size == 0; // true exactly when no elements are
stored
} // end heapIsEmpty

// Returns the number of emails currently stored in the heap.
static int heapSize(const MaxHeap *h) { // h: heap to check
    return h->size; // size field already tracks the element
count
} // end heapSize

// Doubles the heap's array capacity when it becomes full.
static void heapGrow(MaxHeap *h) { // h: heap whose array needs to grow
    int newCapacity = h->capacity * 2; //
double the capacity
    Email *newData = (Email *)realloc(h->data, sizeof(Email) * newCapacity); //
attempt to resize in place or move
    if (newData == NULL) { //
realloc failed (out of memory)
        // The original block (h->data) is still valid and unmodified here;
        // free it before exiting so we don't leak it on the way out.
        fprintf(stderr, "Error: out of memory.\n"); // report the failure to
stderr
        free(h->data); // release the still-valid
original block
        exit(EXIT_FAILURE); // abort the program; nothing
else to do
    } // end if realloc failed

```

```

        h->data = newData;           // point at the (possibly relocated) larger block
        h->capacity = newCapacity;   // record the new capacity
    }                               // end heapGrow

// Swaps the contents of two Email structs in place.
static void swapEmails(Email *a, Email *b) { // a, b: the two emails to swap
    Email temp = *a; // save a's contents temporarily
    *a = *b;         // copy b's contents into a
    *b = temp;       // copy the saved original a into b
}                  // end swapEmails

// Returns 1 if email 'a' should be considered HIGHER priority than
// email 'b' (i.e. 'a' belongs closer to the root of the max-heap).
static int hasHigherPriority(const Email *a, const Email *b) { // a, b: emails to
compare
    if (a->categoryRank != b->categoryRank) { // categories differ: rank
decides outright
        return a->categoryRank > b->categoryRank; // different categories:
higher rank wins
    }
    // Same category: the newer email (larger encoded date) wins.
    return a->dateValue > b->dateValue; // tie-break by date within
the same category
}                                     // end hasHigherPriority

/* Standard array index helpers for a 0-indexed binary heap. */
static int parentIndex(int i) { return (i - 1) / 2; } // index of i's parent
static int leftIndex(int i)   { return 2 * i + 1; }   // index of i's left child
static int rightIndex(int i)  { return 2 * i + 2; }   // index of i's right child

// Moves the element at index i up the heap until the max-heap
// property (parent >= children, by priority) is restored.
static void siftUp(MaxHeap *h, int i) { // h: heap; i: index of the
element to move up
    while (i > 0) { // stop once we reach the root
        int p = parentIndex(i); // p: index of i's parent
        if (hasHigherPriority(&h->data[i], &h->data[p])) { // child outranks its
parent
            swapEmails(&h->data[i], &h->data[p]); // swap them up
            i = p; // continue checking
from the new position
        } else { // child does not
            outrank its parent
                break; // heap property satisfied; stop early
        }
    } // end if/else
} // end while
} // end siftUp

// Moves the element at index i down the heap until the max-heap
// property is restored below it.
static void siftDown(MaxHeap *h, int i) { // h: heap; i: index of the element to
move down
    while (1) { // loop until we break out below
        int left = leftIndex(i); // left: index of i's left child
        int right = rightIndex(i); // right: index of i's right child
        int largest = i; // assume current node is the largest
of the three, then check
    }
}

```

```

        if (left < h->size && hasHigherPriority(&h->data[left], &h->data[largest]))
    { // left child exists and outranks
        largest = left; // left child outranks current largest
    } // end if left child check
    if (right < h->size && hasHigherPriority(&h->data[right],
&h->data[largest])) { // right child exists and outranks
        largest = right; // right child outranks current largest
    } // end if right child check
    if (largest == i) { // neither child outranked the current node
        break; // node is already in the right place; done
    } // end if largest == i
    swapEmails(&h->data[i], &h->data[largest]); // move the larger child up
    i = largest; // continue sifting down from
its new spot
    } // end while
} // end siftDown

// Inserts a new email into the heap, growing the array if necessary,
// and restores the heap property with siftUp.
static void heapInsert(MaxHeap *h, Email e) { // h: heap; e: email to insert (by
value)
    if (h->size == h->capacity) { // array is full
        heapGrow(h); // make room if the array is full
    } // end if array full
    h->data[h->size] = e; // place new element at the next free slot
    siftUp(h, h->size); // restore heap order
    h->size++; // one more element now stored
} // end heapInsert

// Returns a copy of the highest-priority email without removing it.
// Caller must ensure the heap is non-empty.
static Email heapPeek(const MaxHeap *h) { // h: heap to peek at
    return h->data[0]; // root of a max-heap is always the highest priority
element
} // end heapPeek

// Removes and returns the highest-priority email.
// Caller must ensure the heap is non-empty.
static Email heapExtractMax(MaxHeap *h) { // h: heap to extract the max from
    Email top = h->data[0]; // save the root to return later
    h->size--; // shrink the logical size
    h->data[0] = h->data[h->size]; // move the last element to the root
    if (h->size > 0) { // heap still has elements after removal
        siftDown(h, 0); // restore heap order from the root down
    } // end if heap non-empty
    return top; // hand back the removed (formerly
highest-priority) email
} // end heapExtractMax

/* ----- */
/* Parsing helpers */
/* ----- */

// Strips trailing '\r' / '\n' / spaces / tabs from a line read via fgets.
static void trimLine(char *s) { // s: null-terminated string to trim in place
    int len = (int)strlen(s); // len: current length of the string
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r' || // last char
is newline or CR

```



```

        s[len - 1] == ' ' || s[len - 1] == '\t')) { // or a
trailing space/tab
        s[--len] = '\0'; // chop off one trailing character at a time
    } // end while
} // end trimLine

// Returns 1 if 'category' is one of the five sender categories defined
// by the assignment spec, 0 otherwise. Used to validate input before
// it is silently mapped to a default rank.
static int isKnownCategory(const char *category) { // category: string to validate
    return strcmp(category, "Boss") == 0 || // matches "Boss"
        strcmp(category, "Subordinate") == 0 || // or matches
"Subordinate"
        strcmp(category, "Peer") == 0 || // or matches "Peer"
        strcmp(category, "ImportantPerson") == 0 || // or matches
"ImportantPerson"
        strcmp(category, "OtherPerson") == 0; // or matches
"OtherPerson"
} // end
isKnownCategory

// Maps a sender category string to a numeric priority rank.
// Higher number = higher priority (closer to heap root). Subordinate,
// Peer, and ImportantPerson are intentionally tied per the assignment
// spec; ties within a category are broken by date in hasHigherPriority.
// An unrecognized category is treated as lowest priority; the caller
// is responsible for warning the user about unknown categories.
static int categoryToRank(const char *category) { // category: sender category
string
    if (strcmp(category, "Boss") == 0) return 5; // Boss: highest rank
    if (strcmp(category, "Subordinate") == 0) return 3; // Subordinate: tied
"read next" rank
    if (strcmp(category, "Peer") == 0) return 3; // Peer: tied "read
next" rank
    if (strcmp(category, "ImportantPerson") == 0) return 3; // ImportantPerson:
tied "read next" rank
    if (strcmp(category, "OtherPerson") == 0) return 1; // OtherPerson: lowest
defined rank
    return 0; // unknown category: treat as lowest priority rather than crash
} // end categoryToRank

// Returns 1 if the given month/day/year form a plausible calendar
// date, 0 otherwise. Deliberately simple (does not account for leap
// years or exact days-per-month) so it catches clearly bad input
// (month 13, day 0, a negative year) without becoming a full calendar
// validator.
static int isPlausibleDate(int month, int day, int year) { // month/day/year:
parsed date parts
    if (month < 1 || month > 12) return 0; // month must be 1-12
    if (day < 1 || day > 31) return 0; // day must be 1-31 (approximate
upper bound)
    if (year < 1900 || year > 9999) return 0; // year must be a plausible 4-digit
year
    return 1; // all three parts passed their range checks
} // end isPlausibleDate

// Converts a date string "MM-DD-YYYY" into an integer YYYYMMDD so
// that larger values represent more recent dates and can be compared
// numerically without re-parsing. Returns 1 on success, 0 if the

```

```

// string could not be parsed as three integers or failed the range
// check in isPlausibleDate; *outValue is only meaningful when this
// function returns 1.
static int dateToValue(const char *date, long *outValue) { // date: string to
parse; outValue: result
    int month = 0, day = 0, year = 0; // parsed components,
defaulted to 0
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) { // expect exactly 3
integers separated by '-'
        *outValue = 0; // malformed date: caller will warn and treat as oldest
possible
        return 0; // signal failure to the caller
    } // end if sscanf failed
    if (!isPlausibleDate(month, day, year)) { // parsed numbers but out of a sane
range
        *outValue = 0; // treat as oldest possible date
        return 0; // signal failure to the caller
    } // end if implausible date
    *outValue = (long)year * 10000L + (long)month * 100L + (long)day; // encode as
YYYYMMDD
    return 1; // signal success to the caller
} // end dateToValue

// Parses a line of the form:
// <sender category>,<subject line>,<date>
// (the "EMAIL " keyword and following space have already been
// skipped by the caller). Returns 1 on success, 0 if a required
// field is missing. Prints stderr warnings (but still succeeds) for
// recoverable problems: an unrecognized category, a subject line
// long enough to be truncated, an unparseable/out-of-range date, or
// extra comma-delimited fields beyond the three expected ones.
static int parseEmailFields(char *fields, Email *out) { // fields: mutable
comma-delimited text; out: result
    char *category = strtok(fields, ","); // first comma-delimited field
    char *subject = strtok(NULL, ","); // second field
    char *date = strtok(NULL, ","); // third field
    char *extra = strtok(NULL, ","); // anything after the third field is
unexpected

    if (category == NULL || subject == NULL || date == NULL) { // one or more
required fields missing
        return 0; // line was missing one or more required fields
    } // end if missing
field

    if (extra != NULL) { // there was a fourth (or later) comma-delimited field
        fprintf(stderr,
            "Warning: EMAIL line has more than 3 fields; ignoring extra data
starting at '%s'.\n",
            extra); // report the unexpected extra data
    } // end if extra field present

    if (!isKnownCategory(category)) { // category is not one of the five defined
strings
        fprintf(stderr,
            "Warning: unrecognized sender category '%s'; treating as lowest
priority.\n",
            category); // warn that this category will fall back to rank 0
    } // end if unknown category

```

```

    if (strlen(subject) > MAX_SUBJECT - 1) { // subject is longer than the buffer
can hold
        fprintf(stderr,
            "Warning: subject line exceeds %d characters and will be truncated:
's'\n",
            MAX_SUBJECT - 1, subject); // warn that truncation will occur
    } // end if subject too long

    strncpy(out->sender, category, MAX_SENDER - 1); // copy category into the
fixed-size buffer
    out->sender[MAX_SENDER - 1] = '\0'; // guarantee null termination

    strncpy(out->subject, subject, MAX_SUBJECT - 1); // copy subject into the
fixed-size buffer
    out->subject[MAX_SUBJECT - 1] = '\0'; // guarantee null
termination

    strncpy(out->date, date, MAX_DATE - 1); // copy date into the fixed-size
buffer
    out->date[MAX_DATE - 1] = '\0'; // guarantee null termination

    out->categoryRank = categoryToRank(out->sender); // pre-compute for fast
comparisons

    long value = 0; // will hold the encoded date value
    if (!dateToValue(out->date, &value)) { // date could not be
parsed/validated
        fprintf(stderr,
            "Warning: could not parse date '%s' as MM-DD-YYYY; treating as oldest
possible date.\n",
            date); // warn that the date is being treated as the oldest possible
    } // end if date parsing failed
    out->dateValue = value; // store whatever value dateToValue produced (valid or
fallback 0)

    return 1; // signal successful parsing of this EMAIL line
} // end parseEmailFields

/* ----- */
/* Command handlers */
/* ----- */

// Handles an "EMAIL ..." command line: parses it and inserts the
// resulting email into the heap.
static void handleEmail(MaxHeap *h, char *line) { // h: heap to insert into; line:
text after "EMAIL "
    // line currently points at the character right after "EMAIL ".
    Email e; // will hold the parsed email
    if (parseEmailFields(line, &e)) { // parsing succeeded
        heapInsert(h, e); // add the new email to the heap
    } else { // parsing failed (missing field)
        fprintf(stderr, "Warning: malformed EMAIL line skipped (missing field(s)):
%s\n", line); // report and skip
    } // end if/else
} // end handleEmail

// Handles a "NEXT" command: displays the highest-priority email
// without removing it. Calling NEXT twice in a row (no intervening

```

```

// READ) prints the same email both times, as required by the spec.
static void handleNext(MaxHeap *h) { // h: heap to read from
    if (heapIsEmpty(h)) {           // no emails currently queued
        printf("There are no emails to read.\n\n"); // report empty queue
        return;                     // nothing more to do
    }                               // end if empty
    Email top = heapPeek(h);        // look without removing
    printf("Next email:\n");        // header line for the email details
    printf("\tSender: %s\n", top.sender); // print the sender category
    printf("\tSubject: %s\n", top.subject); // print the subject line
    printf("\tDate: %s\n", top.date);    // print the date
    printf("\n");                    // blank line to match sample output
    spacing
}                                   // end handleNext

// Handles a "READ" command: removes the highest-priority email
// (the CEO has dealt with it) with no output. Calling READ twice in
// a row silently discards two emails, as required by the spec. READ
// on an empty queue is a safe no-op.
static void handleRead(MaxHeap *h) { // h: heap to remove from
    if (!heapIsEmpty(h)) {           // only extract if there is something to
        extract
        heapExtractMax(h);           // remove the highest-priority email,
        discarding the result
    }                                 // end if non-empty
}                                    // end handleRead

// Handles a "COUNT" command: prints how many unread emails remain.
static void handleCount(MaxHeap *h) { // h: heap to report the size of
    printf("There are %d emails to read.\n\n", heapSize(h)); // print current
    unread count
}                                    // end handleCount

/* ----- */
/* Main driver */
/* ----- */

int main(int argc, char *argv[]) { // argc: arg count; argv: arg strings (argv[1]
= optional file path)
    FILE *in = stdin; // default to reading commands from standard input

    if (argc > 1) { // a filename was given on the command line
        in = fopen(argv[1], "r"); // otherwise read from the file named on the
command line
        if (in == NULL) { // fopen failed to open the file
            // strerror(errno) gives a specific reason (e.g. "No such file or
            // directory", "Permission denied") instead of a generic message.
            fprintf(stderr, "Error: could not open file '%s': %s\n", argv[1],
strerror(errno)); // report why it failed
            return EXIT_FAILURE; // abort the program with a failure status
        } // end if fopen failed
    } // end if filename given

    MaxHeap heap; // the priority queue used to hold pending emails
    heapInit(&heap); // start with an empty priority queue

    char line[600]; // buffer to hold one line of input at a time
    while (fgets(line, sizeof(line), in) != NULL) { // read one command per line
        trimLine(line); // remove trailing newline/whitespace
    }
}

```

```

    if (line[0] == '\\0') { // line was empty after trimming
        continue; // skip blank lines
    } // end if blank line

    if (strncmp(line, "EMAIL ", 6) == 0) { // line begins with "EMAIL
"
        handleEmail(&heap, line + 6); // skip past "EMAIL " before parsing
    } else if (strcmp(line, "NEXT") == 0) { // line is exactly "NEXT"
        handleNext(&heap); // display the top email
    } else if (strcmp(line, "READ") == 0) { // line is exactly "READ"
        handleRead(&heap); // remove the top email
    } else if (strcmp(line, "COUNT") == 0) { // line is exactly "COUNT"
        handleCount(&heap); // print the current
unread count
    } else { // line did not match any
known command
        fprintf(stderr, "Warning: unrecognized command skipped: %s\\n", line);
// report and ignore it
    } // end if/else if chain
    } // end while reading
lines

    if (in != stdin) { // we opened a real file rather than using stdin
        fclose(in); // only close a real file, never stdin
    } // end if not stdin
    heapFree(&heap); // release the heap's dynamic array

    return EXIT_SUCCESS; // signal successful completion to the OS/shell
} // end main

```